Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 15

Single-Source Shortest Paths (Part 2)

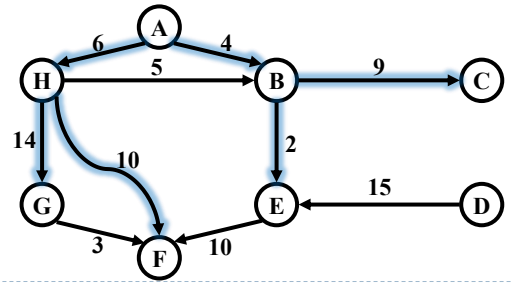
Ομοαφετηριακές Ελαφρύτατες Διαδρομές (Μέρος 2)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

Single-Source Shortest Path
(Ομοαφετηριακές Ελαφρύτατες Διαδρομές)

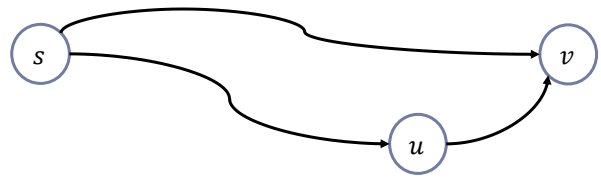
- **Problem:** Given a weighted directed graph G with a weight function $w: \mathrm{E} \to \mathbb{R}$, find the *minimum-weight path* from a given source vertex s to another vertex v
  - Weight of path $p = \langle v_0, v_1, \ldots, v_k \rangle$ is the sum of edges: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$
  - **Shortest path** = path with minimum weight $\boldsymbol{\delta(s, v)}$



Relaxation

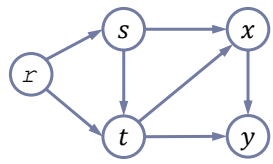
- **Relaxation** (**Χαλάρωση**)**:** Can we improve the shortest-path estimate $d[v]$ for v by going through u and taking (u, v)?

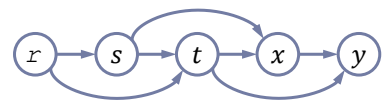
```
Relax(u, v, w) {
   if (d[v] > d[u] + w(u,v)) then
      d[v] = d[u] + w(u,v);
      p[v] = u
}
```



Shortest Paths in a DAG

- **Problem:** Find shortest paths in a DAG
  - Bellman-Ford takes $O(VE)$ time
  - *How can we do better?*
  - **Idea:** use **topological sort** (**τοπολογική ταξινόμηση**)
    - The vertices in *any* path in a DAG have the topologically sorted vertex order
    - Processing vertices in that order, we will do each path in forward order (will never relax outgoing edges before doing all incoming edges)
    - Thus: just one pass!





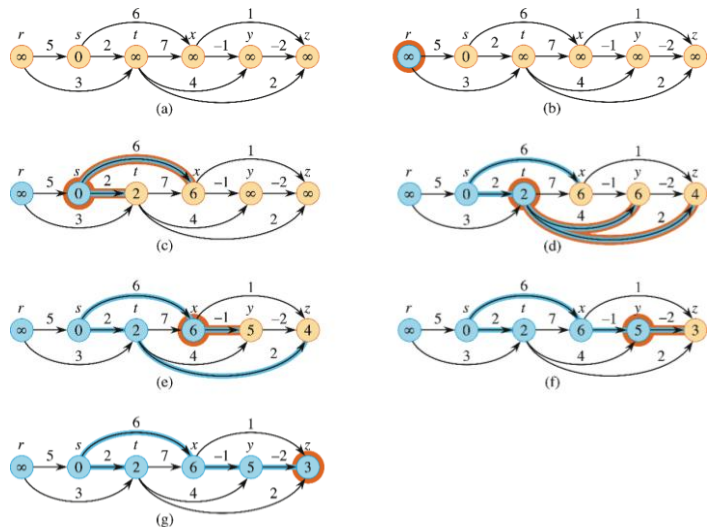
Shortest Paths in a DAG: Pseudocode

```
DAG-SHORTEST-PATHS(G, w, s)
   Topologically sort the vertices G.V
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   for each vertex u in topologically sorted order
      for each vertex v ∈ Adj[u]
         if (d[v] > d[u] + w(u,v))
            d[v] = d[u] + w(u,v);
```

Relaxation Step

What is the total running time? $\Theta(V + E)$

Shortest Paths in a DAG: Example

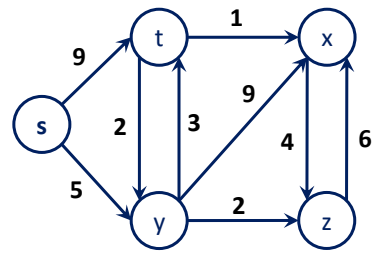


Dijkstra's Algorithm

- If **no** negative edge weights, we can beat Bellman-Ford
- Similar to Prim's algorithm for minimum spanning tree (ελαφρύτατο συνδετικό δένδρο)
  - Grow a tree gradually, advancing from vertices taken from a queue
  - Use a priority queue (ουρά προτεραιότητας) keyed on $d[v]$ (*shortest-path weights*)
- Have 2 sets of vertices
  - $S =$ vertices whose final shortest-path weights are determined
  - $Q =$ priority queue $= V - S$

Dijkstra's Algorithm

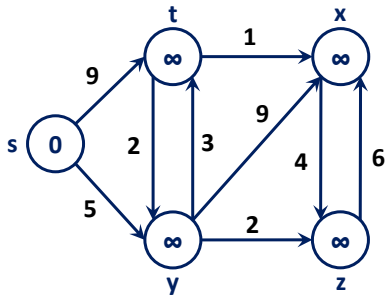
```
Dijkstra(G, w, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   S = ∅;
   Q = G.V;
   while (Q ≠ ∅)
      u = ExtractMin(Q);
      S = S U {u};
      for each v ∈ G.Adj[u]
         if (d[v] > d[u] + w(u,v))
            d[v] = d[u] + w(u,v);
```



Relaxation Step

Note: this is really a call to Q->DecreaseKey()

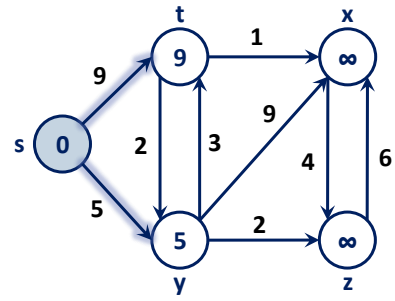
Dijkstra's Algorithm



After initialization

S = ∅

Q = (s,0) (t,∞) (x,∞) (y,∞) (z,∞)

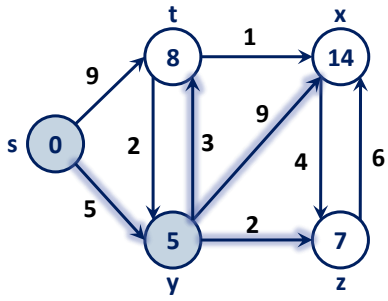


After 1st iteration

S = {s}

Q = (y,5) (t,9) (x,∞) (z,∞)

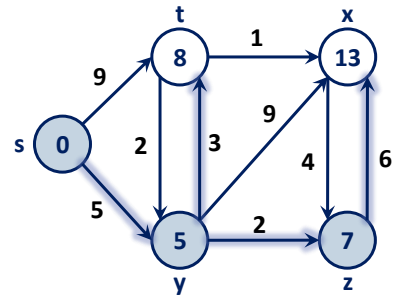
Dijkstra's Algorithm



After 2nd iteration

S = {s, y}

Q = (z,7) (t,8) (x,14)

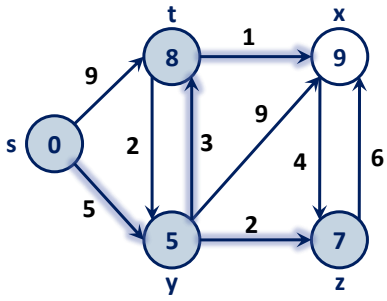


After 3rd iteration

S = {s, y, z}

Q = (t,8) (x,13)

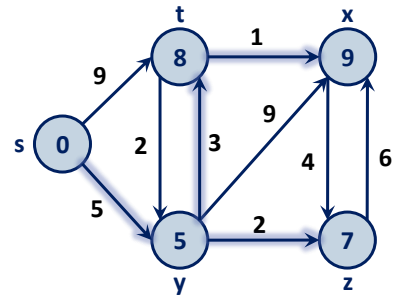
Dijkstra's Algorithm



After 4th iteration

S = {s, y, z, t}

Q = (x,9)



After 5th iteration

S = {s, y, z, t, x}

Q = ∅

Dijkstra's Algorithm

```
Dijkstra(G, w, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   S = ∅;
   Q = G.V;
   while (Q ≠ ∅)
      u = ExtractMin(Q);
      S = S U {u};
      for each v ∈ G.Adj[u]
         if (d[v] > d[u] + w(u,v))
            d[v] = d[u] + w(u,v);
```

How many times is ExtractMin() called? V

How many times is DecreaseKey() called? E

What is the total running time?

With Binary Heap: $O(E \lg V)$

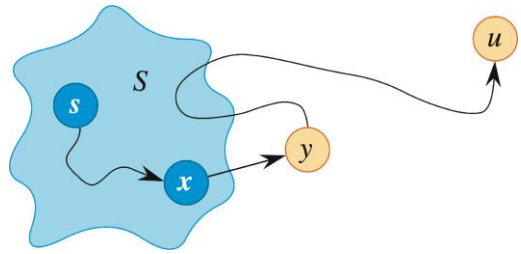
With Fibonacci Heap: $O(V \lg V + E)$

Dijkstra's Algorithm

```
Dijkstra(G, w, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   S = ∅;
   Q = G.V;
   while (Q ≠ ∅)
      u = ExtractMin(Q);
      S = S U {u};
      for each v ∈ G.Adj[u]
         if (d[v] > d[u] + w(u,v))
            d[v] = d[u] + w(u,v);
```

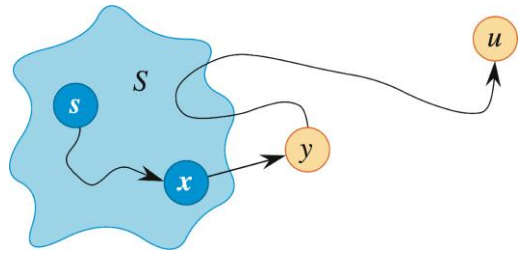
Correctness: we must show that when u is removed from Q, it has already converged

Correctness Of Dijkstra's Algorithm



- Note that $d[v] \geq \delta(s,v)\ \forall v$ (Upper-bound property)
- Let u be first vertex for which $d[u] \neq \delta(s,u)$ when u is added to S
- Let y be first vertex in $V - S$ on shortest path from $s \rightarrow u \Rightarrow d[y] = \delta(s,y)$
  - x is y's predecessor in S and $d[x] = \delta(s,x)$ (*Why?*)
  - When we put x into S, we relaxed $(x,y) \Rightarrow d[y] = \delta(s,y)$ (Convergence property)

Correctness Of Dijkstra's Algorithm



- Note that $d[v] \geq \delta(s, v)\ \forall v$ (Upper-bound property)
- Let u be first vertex for which $d[u] \neq \delta(s, u)$ when u is added to S
- Let y be first vertex in $V - S$ on shortest path from $s \rightarrow u \Rightarrow d[y] = \delta(s, y)$
- But then: $d[u] > \delta(s, u)$ (*Why? Upper-bound property*)
  $= \delta(s, y) + \delta(y, u)$ (*Why? Optimal substructure*)
  $= d[y] + \delta(y, u) \geq d[y]$ (*Why? Only positive edges*)
- But if $d[u] > d[y]$, we wouldn't have chosen u. Contradiction!